MaaS 大模型服务调用 API 文档

1. 接口说明

1.1 大语言模型接口（Chat Completion API）

1.1.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.1.2 请求方法

`POST`

1.1.3 Request

字段	名称	类型	必填	说明
model	模型名称	string	是	模型 ID
messages	消息列表	list<object>	是	对话历史数组，格式为 `[{"role":"user/assistant/system","content":"文本内容"}]`
stream	是否流式	bool	否	是否启用流式响应，默认 `false`
stream_options	流式选项	object	否	流式输出是否包含 token 用量，示例值：{"include_usage": true}
max_tokens	最大token	int	否	最大输出 token 数，默认与模型配置一致（如 doubao 系列默认 12k，deepseek 默认 16k）
stop	停止词	string / list<string>	否	模型遇到 stop 字段所指定的字符串时将停止继续生成。
temperature	温度	float	否	采样温度，控制生成文本时对每个候选

				词的概率分布平滑程度。 生成随机性控制，范围 0.0-2.0，值越大越随机，默认 1.0。
top_p	核采样	float	否	核采样概率阈值，模型会考虑概率质量在 top_p 内的 token 结果
tools	工具	list<object>	否	待调用工具的列表，适用于 function call 场景
thinking	深度思考	object	否	深度思考配置
thinking.show	深度思考展示	bool	否	是否返回深度思考内容 1. 支持的模型范围：模型 FeaturesConfig 属性数组中包含 reasoing
thinking.type	深度思考开关	string	否	深度思考类型: enabled, disabled, auto。 1. enabled：开启思考模式，模型一定先思考后回答。默认值 1. disabled：关闭思考模式，模型直接回答问题，不会进行思考。 1. auto：自动思考模式，模型根据问题自主判断是否需要思考，简单题目直接回答。
reasoning_effort	推理深度	string	否	推理深度，越深度的思考，对于 token 的消耗越大。思考深度支持的值: mini,mal, low, medium, high

1.1.4 Response（流式与非流式）

非流式调用

字段	名称	类型	必填	说明
id	id	string	是	本次请求的唯一标识

model	模型	string	是	本次请求实际使用的模型
choices	输出	object[]	是	本次请求模型输出内容，参考文档：https://www.volcengine.com/docs/82379/1494384#fT1TMaZk
usage	用量	usage	是	
usage.total_tokens	总token数	int	是	
usage.prompt_tokens	输入token数	int	是	
usage.completion_tokens	输出token数	int	是	

流式调用

字段	名称	类型	必填	说明
id	id	string	是	本次请求的唯一标识
model	模型	string	是	本次请求实际使用的模型
choices	输出	object[]	是	本次请求模型输出内容，参考文档：https://www.volcengine.com/docs/82379/1494384#jp88SeXS
usage	用量	usage	是	
usage.total_tokens	总token数	int	是	
usage.prompt_tokens	输入token	int	是	

	数			

1.2 大语言模型接口（Responses API）：

1.2.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.2.2 请求方法

`POST`

1.2.3 Request

字段	名称	类型	必填	说明
model	模型名称	string	是	模型 id 或者路由 id
input	输入内容	string/list<object>	是	参考文档：https://www.volcengine.com/docs/82379/1569618
previous_response_id	上次响应 id	string	否	用于引入上一轮请求的输入和回答内容
max_output_tokens	最大输出 token	int	否	控制模型输出的最大长度
store	存储开关	bool	否	是否存储响应 1. false：不存储，每次请求需携带完整的上下文信息 1. true：存储，每次请求只需携带最新输入+previous_response_id，模型通过 id 关联上下文信息
				是否返回深度思考内容 1. 支持的模型范围：模型 FeaturesConfig 属性数组中包含 reasoing
stream	是否	bool	否	响应内容是否流式返回

	流式			
reasoning	推理深度	object	否	推理深度配置
reasoning.effort	推理深度	string	否	推理深度 1. 支持的模型范围：模型 FeaturesConfig 数组中包含 reasoing-effort 1. 可选值来源：模型 ReasoningConfig 属性中 Effort.Type 数组 1. 默认值：模型 ReasoningConfig 属性中 Effort.DefaultType
thinking	深度思考	object	否	深度思考配置
thinking.type	深度思考开关	string	否	深度思考开关 1. 支持的模型范围：模型 FeaturesConfig 属性数组中包含 reasoing-switch 1. 可选值来源：模型 ReasoningConfig 属性的 Switch.Type 数组 1. 默认值：模型 ReasoningConfig 属性的 Switch.DefaultType
temperature	温度	float	否	采样温度，控制生成文本时对每个候选词的概率分布平滑程度
top_p	核采样	float	否	核采样概率阈值，模型会考虑概率质量在 top_p 内的 token 结果
tools	工具	list<object>	否	待调用工具的列表，适用于 function call 场景，参考文档：https://www.volcengine.com/docs/82379/1569618
max_tool_calls	最大工具调用	int	否1.	最大工具调轮次。在工具调用达到此限制次数后，提示模型停止更多工具调用并进行回答。

	次数		1.	注意：该参数为尽力而为（best effort）机制，不保证成功，最终调用次数会受模型推理效果、工具返回结果有效性等因素影响。

1.2.4 Response（流式与非流式）

非流式调用

非流式调用一次输出完整的 output

字段	名称	类型	必填	说明
id	响应 id	string	是	本次请求的唯一标识
model	模型	string	是	本次请求实际使用的模型
output	输出	object[]	是	本次请求模型输出内容，参考文档：https://www.volcengine.com/docs/82379/1783703
previous_response_id	上次响应 id	string	是	本次请求时传入的历史响应 ID。
status	响应的状态	string	是	生成响应的状态。 1. completed：响应已完成。 1. failed：响应失败。 1. in_progress：响应中。 1. incomplete：响应未完成。
store	存储开关	bool	是	是否储存生成的模型响应
usage	用量	usage	是	
usage.total_tokens	总 token 数	int	是	

usage.input_tokens	输入token数	int	是	
usage.output_tokens	输出token数	int	是	

流式调用

流式调用通过 SSE 事件流式返回 output，事件详情参考文档：https://www.volcengine.com/docs/82379/1599499

事件	名称	必须	字段	说明
response.created	响应创建	是	1. reponse：object，创建状态的响应，结构和非流式调用返回一致 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是response.created	
response.in_progress	响应进行中	是	1. reponse：object，进行中状态的响应，结构和非流式调用返回一致 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是response.in_progress	
response.completed	响应完成	是	1. reponse：object，已完成状态的响应，结构和非流式调用返回一致 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是response.completed	

response.failed	响应失败	否	1. reponse：object，失败状态的响应，结构和非流式调用返回一致 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是response.failed	当响应失败时才会触发该事件
response.incomplete	响应未完成	否	1. reponse：object，未完成状态的响应，结构和非流式调用返回一致 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是response.incomplete	当响应以未完成状态结束时才会触发该事件
response.output_item.added	输出项添加	是	1. item：object，模型输出内容，结构和非流式调用返回的 output 参数一致 1. sequence_number：int，事件的序列号 1. output_index: int，输出项的索引号 1. type：string，事件类型，总是response.incomplete	
response.output_item.done	输出项完成	是	1. item：object，模型输出内容，结构和非流式调用返回的 output 参数一致 1. sequence_number：int，事件的序列号 1. output_index: int，输出项的索引号 1. type：string，事件类型，总是response.incomplete	

response.output_text.delta	文本片段新增	否	1. delta：string，新增内容 1. content_index: int，所属内容块的索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是 response.incomplete	当有新增文本片段时触发
response.output_text.done	文本内容完成	否	1. text：string，完整内容 1. content_index: int，所属内容块的索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是 response.incomplete	文本内容完成时触发
response.function_call_arguments.delta	函数调用参数片段	否	1. delta：string，新增内容 1. content_index: int，所属内容块的索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是	存在函数调用参数片段时触发

			response.incomplete	
response.function_call_arguments.done	函数调用参数完成	否	1. text：string，完整内容 1. content_index: int，所属内容块的索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是 response.incomplete	当函数调用参数完成时触发
response.reasoning_summary_text.delta	文本片段新增	否	1. delta：string，新增内容 1. summary_index: int，输出项内，推理总结部分的子索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号 1. type：string，事件类型，总是 response.incomplete	当存在思维链新增文本时触发
response.reasoning_summary_text.done	文本内容完成	否	1. text：string，完整内容 1. summary_index: int，输出项内，推理总结部分的子索引 1. item_id: string, 所属输出项的唯一 ID 1. output_index: int，所属输出项的索引号 1. sequence_number：int，事件的序列号	思维链文本完成时触发

		1.	type：string，事件类型，总是 response.incomplete	

1.3 Embedding 模型接口：

1.3.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.3.2 请求方法

`POST`

1.3.3 Request

字段	名称	类型	必填	说明
model	模型名称	string	是	对应模型 id
input	输入	String or list<string>	是	字符串或数组类型

1. 对于支持视觉理解的模型，可以在 input 里传入多模态数据
    1. 判断模型是否支持视觉理解：模型的 FeaturesConfig 属性字段里有“embedding-vision”
    1. 对于不支持视觉理解的模型：不建议传入多模态数据，模型无法处理会报错

1.3.4 多模态请求体数据结构示例：

```JSON
{
    "model": "d2tfsttng3js73fulmm0",
    "input": [
        [
            {
                "type": "video_url",
                "video_url": {
                    "url":
"https://example.com/doc_video/ark_vlm_video_input.mp4"
                }
            },
            {
                "type": "image_url",
                "image_url": {
                    "url":
"https://example.com/doc_image/tower.png"
```

```
                }
            },
            {
                "type": "text",
                "text": "视频和图片里有什么"
            }
        ]
    ]
}
```

1.3.5 Response

字段	名称	类型	必填	说明
data	结果	list<Embedding>	是	数组，对应输入 input 的元素
data.embedding	向量	float32[]	是	向量值
data.index	index	int	是	对应输入 input 的 index
usage	用量	object	是	-
usage.total_tokens	总 token 数	int	是	-
usage.prompt_tokens	输入 token 数	int	是	-

1.4 Rerank 模型对接

1.4.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.4.2 请求方法

`POST`

1.4.3 Request

字段	名称	类型	必填	说明

model	模型名称	string	是	对应模型管理的 id
query	查询	string	是	查询语句
documents	文档列表	list<string>	是	
top_n	返回 top_n	int	否	默认全返回

1. 对于支持多模态的模型，可以在 input 里传入多模态数据
    1. 判断模型是否支持视觉理解：模型的 FeaturesConfig 属性字段里有“rerank-vision”
    1. 对于不支持视觉理解的模型：不建议传入多模态数据，模型无法处理会报错
    1. 多模态请求体数据结构示例：

```JSON
{
    "model": "d2tfsttng3js73fulmm0",
    // query 支持多模态数据
    "query": [
        {
            "type": "image_url",
            "image_url": {
                "url": "https://ark-project.tos-cn-
beijing.volces.com/doc_image/tower.png"
            }
        },
        {
            "type": "text",
            "text": "召回相似图片"
        }
    ],
    // document 也支持多模态数据
    "documents": [
        [
            {
                "type": "image_url",
                "image_url": {
```

```
                    "url": "https://ark-project.tos-cn-
beijing.volces.com/doc_image/tower.png"
                }
            },
            {
                "type": "text",
                "text": "伦敦塔"
            }
        ]
    ]
}
```

1.4.4 Response

字段	名称	类型	必填	说明
results	结果	list<result>	是	
results.document	文档	string	否	
results.relevance_score	相关度	float	是	重排后的相关度
results.index	index	int	是	对应输入 documents 的 index
usage	用量	object	是	
usage.total_tokens	总 token 数	int	是	
usage.prompt_tokens	输入 token 数	int	是	

1.5 视觉模型对接

1.5.1 生图片模型

1.5.1.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.5.1.2 请求方法

`POST`

1.5.1.3 Request

字段	名称	类型	必填	说明
model	模型名称	string	是	对应模型管理的 id
prompt	提示词	string	是	
image	图片	string / list<string>	否	图生图模型参数
size	图片分辨率	string	否	生成图片的分辨率，比如 1024x1024
seed	种子值	int	否	随机数种子，用于控制模型生成内容的随机性
guidance_scale	引导尺度	float	否	模型输出结果与 prompt 的一致程度，生成图像的自由度，又称为文本权重
response_format	图片格式			生成图片的格式 1. url: 返回图片下载链接 1. b64_json: 以 Base64 编码字符串的 JSON 格式返回图像数据
user	用户	string	否	用于观测

1.5.1.4 Response

字段	名称	类型	必填	说明
data	数据	list<object	是	

		>		
data.url	图片 url	string	否	图片的 url 信息，当 response_format 指定为 url 时返回
data.b64_json	图片 b64	string	否	图片的 base64 信息，当 response_format 指定为 b64_json 时返回
usage	用量	usage	是	
usage.generated_images	生成图片张数	int	是	
usage.total_tokens	总 token 数	int	是	
usage.prompt_tokens	输入 token 数	int	是	
usage.completion_tokens	输出 token 数	int	是	

1.5.2 视频生成模型对接

1.5.2.1 创建视频生成任务

1.5.2.1.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.5.2.1.2 请求方法

`POST`

1.5.2.1.3 Request

字段	名称	类型	必填	说明

model	模型名称	string	是	对应模型管理的 id
content	输入内容	list<object>	是	
content.type	内容类型	string	是	枚举值： 1. text: 文本提示词 1. image_url：图片信息
content.text	文本内容	string	否	输入给模型的文本内容，描述期望生成的视频。type 为 text 时必填。
content.image_url	图片内容	string	否	输入给模型的图片内容，用于图生视频。type 为 image_url 时必填
content.role	图片角色	string	否	图片的位置或用途，图生视频时必填。枚举值： 1. first_frame：首帧图片 1. last_frame：尾帧图片 1. reference_image：参考图
generate_audio	声音	bool	否	控制生成的视频是否包含与画面同步的声音。部分模型支持，默认包含
resolution	分辨率	string	否	生成视频的分辨率，比如，720p
ratio	比例	string	否	生成视频的比例，比如，16:9
duration	时长	int	否	生成视频时长，单位：秒
camerafixed	固定镜头	bool	否	固定镜头参数，特定模型才支持

seed	种子值	int	否	随机数种子，用于控制模型生成内容的随机性

1.5.2.1.4 Response

字段	名称	类型	必填	说明
id	任务 id	string	是	

1.5.2.2 查询视频生成任务

通过“创建视频生成任务”得到的任务 id，可以查询视频生成任务执行结果

1.5.2.2.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.5.2.2.2 请求方法

`GET`

1.5.2.2.3 Request

字段	名称	类型	必填	说明
filter.task_id	task_id 条件	string	否	根据 task_id 过滤查询任务
option.page_num	页码	int	否	分页查询页码
option.page_size	页大小	int	否	分页查询页大小
option.sort	排序条件	list<object>	否	
option.sort.sort_field	排序字段	string	否	
option.sort.sort_order	排序顺序	string	否	排序顺序: 1. desc：降序排列 1. asc：升序排列

1.5.2.2.4 Response

字段	名称	类型	必填	说明
total	总数	int	是	任务总数量
items	任务列表	list<object>	是	任务列表
items.status	任务状态	string	是	任务状态
items.data	数据	list<object>	是	
items.data.url	视频 url	string	否	生成视频的 url 信息
items.usage	用量	usage	是	
items.usage.total_tokens	总 token 数	int	是	
items.usage.prompt_tokens	输入 token 数	int	是	
items.usage.completion_tokens	输出 token 数	int	是	

1.5.2.3 取消视频生成任务

通过“创建视频生成任务”得到的任务 id，可以取消视频生成任务。

1.5.2.3.1 Request

字段	名称	类型	必填	说明
task_id	任务 id	string	是	创建视频生成任务返回的任务 id，path 参数

1.5.2.3.2 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.5.2.3.3 请求方法

`DELETE`

1.5.2.3.4 Request

字段	名称	类型	必填	说明
filter	过滤条件	object	否	分页查询过滤条件
filter.task_id	task_id 条件	string	否	根据 task_id 过滤查询任务
option	查询选项			
option.page_num	页码	int	否	分页查询页码
option.page_size	页大小	int	否	分页查询页大小
option.sort	排序条件	list<object>	否	
option.sort.sort_field	排序字段	string	否	
option.sort.sort_order	排序顺序	string	否	排序顺序: 1. desc：降序排列 1. asc：升序排列

1.5.2.3.5 Response

字段	名称	类型	必填	说明
total	总数	int	是	任务总数量
items	任务列表	list<object>	是	任务列表
items.status	任务状态	string	是	任务状态

items.data	数据	list<object>	是	
items.data.url	视频 url	string	否	生成视频的 url 信息
items.usage	用量	usage	是	
items.usage.total_tokens	总 token 数	int	是	
items.usage.prompt_tokens	输入 token 数	int	是	
items.usage.completion_tokens	输出 token 数	int	是	

1.6 语音模型对接

通过“创建视频生成任务”得到的任务 id，可以查询视频生成任务执行结果。

1.6.1 录音文件识别模型对接

可以参考 OpenAI 语音转录接口文档：https://platform.openai.com/docs/api-reference/audio/createTranscription

1.6.1.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.6.1.2 请求方法

`POST`

1.6.1.3 Request

字段	名称	类型	必填	说明
model	模型名称	string	是	对应模型管理的 id
file	文件对象	file	否	用于转录的音频文件对象（不是文件名）

url	文件 url	string	否	文件 url。file 和 url 二选一
language	语言	string	否	输入音频的语言
prompt	提示词	string	否	部分模型支持
user	用户	string	否	用于观测

1.6.1.4 Response

字段	名称	类型	必填	说明
duration	持续时间	float	是	输入音频的持续时间，单位：秒
text	转录文本	string	是	整个音频输入的转录文本
segments	文本片段	list<object>	否	带有时间和 speaker 的文本片段，文本片段可以理解成音频里的一句话，部分模型支持
segments.start	开始时间	float	否	文本片段的开始时间，单位：秒
segments.end	结束时间	float	否	文本片段的结束时间，单位：秒
segments.text	内容	string	否	文本片段的内容
segments.speaker	说话人	string	否	文本片段的说话人信息

1.6.2 流式语音模型对接

统一双向流式语音生成模型（TTS）和语音识别模型（ASR）接口为 /realtime，使用 websocket 协议

1. 通过不同的事件和事件属性定义区分不同模态（文本/语音）的输出和输出
1. 更多信息可以参考 OpenAPI Realtime API 文档：https://platform.openai.com/docs/api-

reference/realtime

1.6.2.1 URL 请求地址

私有化部署场景请联系平台实施同学提供。

1.6.2.2 请求方式

WS /realtime

1. 协议：websocket

1.6.2.3 Query

请求 query 参数

参数	名称	必填	示例	说明
model	模型 id	是 1.	示例：ws://aigw-proxy:6790/v1/realtime?model=<model_id>	调用模型的 id
token	token	否 1.	示例：ws://aigw-proxy:6780/v1/realtime?model=<model_id>&token=<token>	可选，用于对外开放语音模型调用

1.6.2.4 事件

事件分为客户端发送的事件和服务端响应的事件，下面分别介绍。

客户端

客户端事件是指由客户端发送给服务端的事件

分类	事件	名称	必须	示例	说明
通用	session.update	初始化（更新）session	是	{ "type": "session.update", "session": { "type": "realtime", "model": "gpt-realtime", "output_audio_sample_rate": 24000, "voice": "zh_female_shuangkuaisis	初始化（更新）session

				i_moon_bigtts", "output_audio_format": "mp3", } }	

TTS 输入	conversation.item.create	会话 item 创建	否	{
				"type": "conversation.item.create",
				"item": {
				"type": "message",
				"role": "user",
				"content": [
				{
				"type": "input_text",
				"text": "hi"
				}
				]
				},
				"event_id": "b904fba0-0ec4-40af-8bbb-f908a9b26793",
				}
					客户端发送文本
					适用于 TTS 场景

ASR 输入	input_audio_buffer.append	客户端发送语音内容	否	{
				"type": "input_audio_buffer.append",
				"audio": "<base64_data>",
				}
					客户端增量发送语音内容
					适用于 ASR 场景

	input_audio_buffer.commit	客户端完成语音内容发送	否	{
				"type": "input_audio_buffer.commit",
				"response": {
				"output_modalities": ["text"]
				}
				}
					客户端完成语音内容发送
					如果是支持双向流的模型，服务端会在 commit 之前流式响应 ASR 结果

				1.	适用于 ASR 场景

通用	response.create	创建响应	是	{
				"type": "response.create",
				"response": {
				"output_modalities": ["audio"]
				}
				}
					1. 客户端请求服务端响应，并执行响应模态

服务端

服务端事件是指服务端响应给客户端的事件

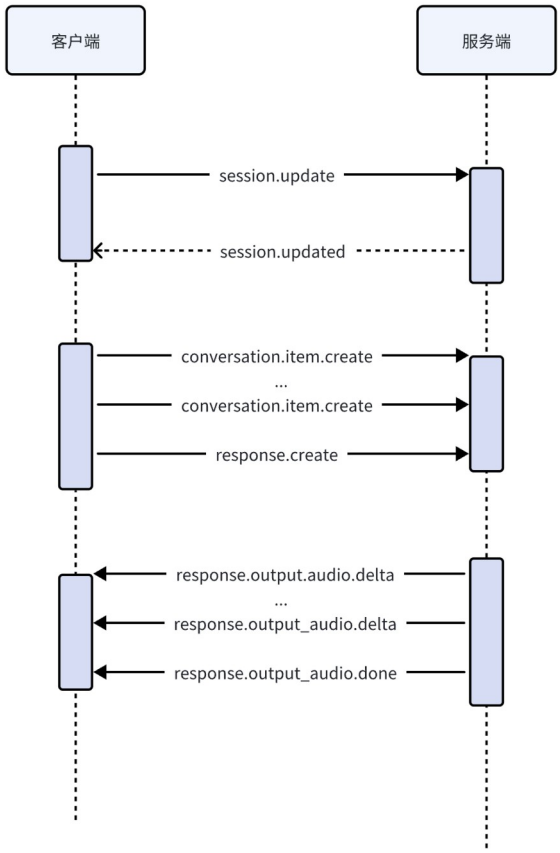
分类	事件	名称	必须	示例	说明

通用	session.updated	更新 session 成功	是	{
				"event_id": "event_vzMEzJOTuWV4zGk0LIO0p",
				"type": "session.updated",
				"session": {
				"id": "sess_cSpNpX3Waw9VAMdFDcqAR",
				"object": "realtime.session",
				"model": "gpt-realtime",
				"type": "realtime",
				"voice": "zh_female_shuangkuaisisi_moon_bigtts",
				"output_audio_format": "mp3",
				"output_audio_sample_rate": 24000,
				"output_audio_channel": 1
				}
				}
					1. 响应客户端发送的 session.update 事件
					1. 适用于所场景

TTS 输出	response.output_audio.delta	服务端增量响应语音内容	否	{ "event_id": "event_psKUevHBqYggUTTpaI3ol", "type": "response.output_audio.delta", "item_id": "item_LB0qzbYm5gWtrbnocyyDy", "delta": "<audio_data>" }	服务端响应语音内容 适用于 TTS 场景
	response.output_audio.done	服务端完成语音内容响应	否	{ "event_id": "event_28VhJBlUwBz5qtcihlx2H", "type": "response.output_audio.done", "item_id": "item_LB0qzbYm5gWtrbnocyyDy" }	服务端完成语音内容响应 适用于 TTS 场景
ASR 输出	response.output_text.delta	服务端增量响应文本内容		{ "event_id": "event_MDFqnN0i5CmkXDnkuXwqz", "type": "response.output_text.delta", "item_id": "item_81V1gBfT2sS1o9Xatsv6q", "delta": "<text>" }	服务端响应文本内容 适用于 ASR 场景
	response.output_text.done	服务端完成文本内容响应		{ "event_id": "event_zztOWTviWhUCANlMmBEab", "type": "response.output_text.done",	服务端完成文本内容响应 适用于 ASR 场景

				"item_id": "item_81V1gBfT2sS1o9Xatsv6q", "text": "<text>" }	
通用	error	服务端响应错误	否	{ "event_id": "event_Cff8jswZNLRDBKFHiTtnJ", "type": "error", "error": { "type": "invalid_request_error", "code": "invalid_value", "message": "Invalid modalities: ['audio', 'text']. Supported combinations are: ['text'] and ['audio'].", "param": "session.output_modalities" } }	1. 服务端返回错误信息，可能是服务端或者客户端的原因 1. 注意：大多数错误是可以恢复的，会话会保持打开状态

1.6.2.5 TTS (语音合成)对接流程



1. response.output.audio.delta 是流式响应，可能在 response.create 事件发送前返回

事件示例

```
Python

# 客户端初始化（更新）session
{
    "type": "session.update",
    "session": {
        "type": "realtime",
        "model": "gpt-realtime",
        "output_audio_sample_rate": 24000,
        "voice": "zh_female_shuangkuaisisi_moon_bigtts",
        "output_audio_format": "mp3",
    }
}

# 服务端响应 session 更新成功
{
  "event_id": "event_vzMEzJOTuWV4zGk0LlO0p",
  "type": "session.updated",
  "session": {
```

```
    "id": "sess_cSpNpX3Waw9VAMdFDcqAR",
    "object": "realtime.session",
    "model": "gpt-realtime",
    "type": "realtime",
    "voice": "zh_female_shuangkuaisisi_moon_bigtts",
    "output_audio_format": "mp3",
    "output_audio_sample_rate": 24000,
    "output_audio_channel": 1
  }
}

# 客户端发送语音生成文本。一次发送 conversation.item.create，多次发送
input_text.append
{
  "type": "conversation.item.create",
  "item": {
    "type": "message",
    "role": "user",
    "content": [
      {
        "type": "input_text",
        "text": "hi"
      }
    ]
  },
  "event_id": "b904fba0-0ec4-40af-8bbb-f908a9b26793",
}

# 客户端完成语音生成文本发送，并设置输出模态为音频
{
    "type": "response.create",
    "response": {
        "output_modalities": ["audio"]
    }
}

# 服务端开始响应语音生成音频。可以多次返回 response.audio.delta
{
  "event_id": "event_psKUevHBqYggUTTpaI3ol",
  "type": "response.output_audio.delta",
  "item_id": "item_LB0qzbYm5gWtrbnocyyDy",
  "delta":
"SUQzBAAAAAAAI1RTU0UAAAAPAAADTGF2ZjU4Ljc2LjEwMAAAAAAAAAAAAAAA//OEx
AAfW1owDHmGORz+GGjicLFJQvY17s8mTBwsgACBhBCIQJ3ad27uiIhcRHc93rumnER
```

```
E3dz6+iF6IiP7vu7pxERPd3d36fEQuf+7/uIT0RELd3dz9P/Qvm7/XRET0RAMyDvv+
+iE+j1zLnvERCc0d4nQ4QAf/AP/
zDxE4AAeHh+GZj+AGckfihwO+iDLCBjLX2iPyzyAoelboy2GY7LZLS2ApNxTRBKItB
VsiN1N6dwQ67RynQLPeJisq7W+//
OExDUk7Co0KNmGuSt5RDHB3KgubnA6mhEdKQG7lERXSHxIZFu/
vacqVyJjRqVrIopHe9yxayGqO2un1ThFtazmaVHOK7bUhRQ7CHNIufTP6QYO9SIiHw
0sLz1oIxDllRc2K+s62WcgjCCVISBk8ObEDmADiZy6aaQUUaF2OKPDWmaZupkzDkSM
+7ShxpqPMfF2+KwiYKoAsxBapCCqs1RyuQbKVnqtlxBW5PnCeP9dUn0iwox3ILujvl
TDHtIc//
OExFQim1I4SNmGXSDaruhakbArtkTMFpnfi3Ml280uye1QtcOp0z19iPiXGqh1OMmH
2GrMkKR6cyEJka4PZz//
dIoVQjoxHGQzGyYDL6QNfyRyelvS+zLrVXdFfyyr6LURTTDy9YiWbqcIrz7CDmPHNr
ENY+OqWSm4tvfK7y7+zQnJdMvbJmGVSVdDtEbmyrbFT0xE8RiFLh4SHKKfenCIyHPQ
MDzMYlKTI8MXTQ61OOiVSUyfcGUNyOdc5Gae//
OExHwlu8o0yOGG3UsJ8x2oJ1LE8zzOKpmS9HctyJNrSPIBYgYUZweJNEMzAzCxCMJX
oCWYmJyiHDMmsorGCfbqT3ar1Hg0gmugZSioB6GCrUGLUEgJSBYIKsEjHiYoDD6CmL
MtLTC2ztMlT1CvGiNh1KwQTpsZIokTTpHbBRJezDspjrhwNjhvWlIhzVDMosCuFcfD
Q/VqHPKqKjY48Qnw/kY0rqZCNCSkJprSMjkUjqEXPkxFx4N0Hhz6DMd+//
OExJglQ/
o0ANpGOWxj5+AQiNSYtnKZRQ0vZXL7N2zSU1TPljPKv3aRhc2WonqZ66xFlvT9Px0u
8x217qvBiXhprZqJ8PJ91mN4E7353vn7ps0mjGVKO7f3U4xybf+SuX9+fKNnGiruGz
fHeKISmlUL+fH0wgyOtmayExTtt4xizc8+r/zY3/7jdqdmOz9bM/T1tdtf/
38jU62PMnjwDiT5qckoSiVWzNevx7StWKASgBDh7Ah8JIGbhgbt+sZn//
OExLYmc8o4wVswAQxeGzF4XPjrs2gdq3qOWjJKZFj3N6tsPY8vQ/
AMWBVNPjWl7sVXXX+XEVyYGDxn3kc1blLOIS3jpxYWmNMGRyMvXFqSzIofldNUl7r1
LA88vusiUoVxOiiGdB8ktPbG5mVxS9FXRryeAspl6b+e6e3P01J27hamLl2alkmuTD
jWXcbCsMyVrsrvT0R3lZxt2s/
q2p6Yubp7Eoxl8buxZ7l30jW3QXe78tpree71uZzt8qb/"
}
...
{
  "event_id": "event_f1pbWDt3KMfZM33EDsnHo",
  "type": "response.output_audio.delta",
  "item_id": "item_LB0qzbYm5gWtrbnocyyDy",
  "delta":
"//OExPQtA/IwCOsMWdOvDybYea5c6JL6ZPUuiZMz4LVSmte1bmnq7Tm3hqOPFnE1p
+3LJ6/N3332D4lkfzDk9Yw+iWa7bJQHK4ttfX/
pCEJxue0QI18zWTdBtiG7ZOSV9mpROLJVr0/7vj39l86Xu3/60dzug1s8l05Z+o4uN
RHEhCmIoimCIHCADkuigDlAGeuQSzgSTYRC0gHhbdAtCVS8sUL0NEfadPrzl5CmFhQ
Vz+BFSN2o8gaXKfSgYxa7//
OExPMsXCIwCNsMNYOmOQZiju6mwihb2MWTIQeiacSRARIpNBoWqa74b6Sjp2kRWyF1
GtJd6g/y7ffqju+t9hzn+/E6N0/
TNx8c5z1VvUkqH1OTi1XsfexCG1HmkrKKy8fw7UicRktrzL19J7yR0/HfH3ZQ2W/
vv45j6/
```

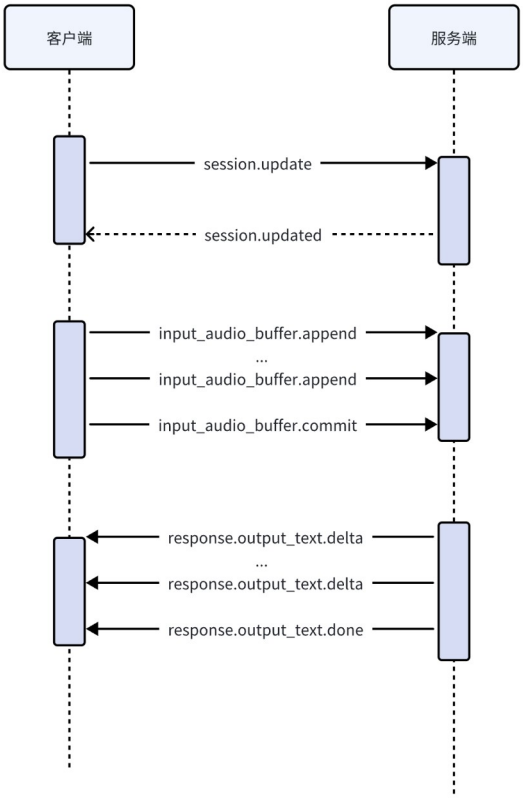
```
ai3ZpLgrEqMciTIRABQoWNB4ZHiJlreOOsO6MNFRWdPpSwObA7gzNDMgWiUxEVozT4
dnpmcq8dPaLmy4jZOgvTU2OT//
OExPQt3DIwCOsMXNSlG12bXDtlF0lZRtqbdij3rWUsPyBXV/
dYiqqRnGfCsh5MjGYu5nMssDZs3k9ucqP4rTme3eU0IeXKn7WYzYgyEvr2uq/
Te6Q9VWN8rG8Luejt29ahmbHamKzsUg2KzbMVDumu9lz329/09b9B5g82ASylTEEUT
KrKvkyPNyTIFDVdsIZ1Iok/1uGX9ipQCS/
cFQJEkjSVkSOHAJZFKSKLVZpHTQUJzzjEQkjlVvY2TgZK//
OExO8r7CIwANsMeawMpcMKJgYBJmZgwE0AgJj/Vmb/6GFGq/
VY1AVVV1CgIlV4GCgImAQEf/6qpMqwMBNVLjNqArP6AgJMwUBVSYCZoBCj/
+H7Mx7H7MevGZjVVP+qAgJUKAqqrhgESvGZgI0K8KOyO/
IB1AngKoE0CRBkAjYFQGeKeNIdouw0SADnHISM2TOO09j3Ow/
z8RaRSSRXl2pGNgY2uBDpeVNVNGnGgQGBChIoSBAYgWIFCRQkCFiB//OExPApA/
I01MGGnWIAiyiyjzLhna82Wd6mpKLKLNONOLKPMLLKNKLMPMLLKdni8//
ynZ2aakososo4040osw8o404so8w8wosouOjpL/8tZZZZbFlBUJbLLKlsss//
5ZZFZZQQMECChgYIGEDggQMEDBVHgKqoMLU22kxBTUUzLjEwMKqqqqqqqqqqqqqq
qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq//
OExP8uq/
U8AHmHPaqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
qqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqqq"
}

# 服务端完成语音生成音频响应
{
  "event_id": "event_28VhJBlUwBz5qtcihlx2H",
  "type": "response.output_audio.done",
  "item_id": "item_LB0qzbYm5gWtrbnocyyDy"
}
```


1.6.2.6 ASR (语音识别)对接

流程

梁瑾 2918



1. response.output_text.delta 是流式响应，可能在 input_audio_buffer.commit 事件发送前返回

事件示例

```
Python
# 客户端初始化（更新）session
{
    "type": "session.update",
    "session": {
        "type": "realtime",
        "model": "gpt-realtime",
        "input_audio_format": "wav",
        # "input_audio_codec": 1, # channel
        # "input_audio_sample_rate": 24000,
        # "input_audio_bits": 4,
    }
}

# 服务端响应 session 更新成功
{
  "event_id": "event_09bHY3v38oPNboIqymLGX",
  "type": "session.updated",
  "session": {
    "id": "sess_wp8WqGgdxjHDk9HisBXty",
    "object": "realtime.transcription_session",
```

```
    "model": "gpt-realtime",
    "type": "realtime",
    "input_audio_format": "wav",
    #"input_audio_transcription": {
    #  "model": "bigmodel"
    #}
  }
}

# 客户端发送语音识别音频。可以多次发送 input_audio_buffer.append
{
    "type": "input_audio_buffer.append",
    "audio": "<base64_data>",
}

# 客户端完成语音识别文本发送，并设置输出模态为文本
{
    "type": "input_audio_buffer.commit",
    "response": {
        "output_modalities": ["text"]
    }
}

# 服务端响应语音识别文本。可以多次返回
conversation.item.input_audio_transcription.result
{
  "event_id": "event_MDFqnN0i5CmkXDnkuXwqz",
  "type": "response.output_text.delta",
  "item_id": "item_81V1gBfT2sS1o9Xatsv6q",
  "delta": "All right.Um, you probably heard about Deep Sea.Uh,
everyone has for some reason. Uh, it's just kind of crazy, so.",
  # "transcript": "All right.Um, you probably heard about Deep
Sea.Uh, everyone has for some reason. Uh, it's just kind of crazy,
so.",
  #"words": [
  #  {
  #    "word": "All",
  #    "start": 1.512,
  #    "end": 1.592
  #  },
  #  ...
  #  {
  #    "word": "so",
  #    "start": 9.412,
```

```
  #    "end": 9.492
  #  }
  #]
}

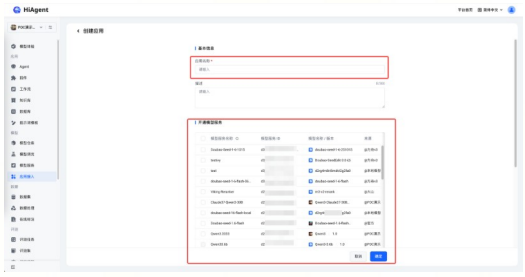
# 服务端完成语音识别文本响应
{
  "event_id": "event_zztOWTviWhUCANlMmBEab",
  "type": "response.output_text.done",
  "item_id": "item_81V1gBfT2sS1o9Xatsv6q",
  "text": "All right. Um, you probably heard about Deep Sea. Uh,
everyone has for some reason. Uh, it's just kind of crazy, so.",
  # "transcript": "All right. Um, you probably heard about Deep
Sea. Uh, everyone has for some reason. Uh, it's just kind of
crazy, so.",
  #"words": [
  #  {
  #    "word": "All",
  #    "start": 1.512,
  #    "end": 1.592
  #  },
  #  ...
  #  {
  #    "word": "so",
  #    "start": 9.412,
  #    "end": 9.492
  #  }
  #]
}
```

1.7 身份验证

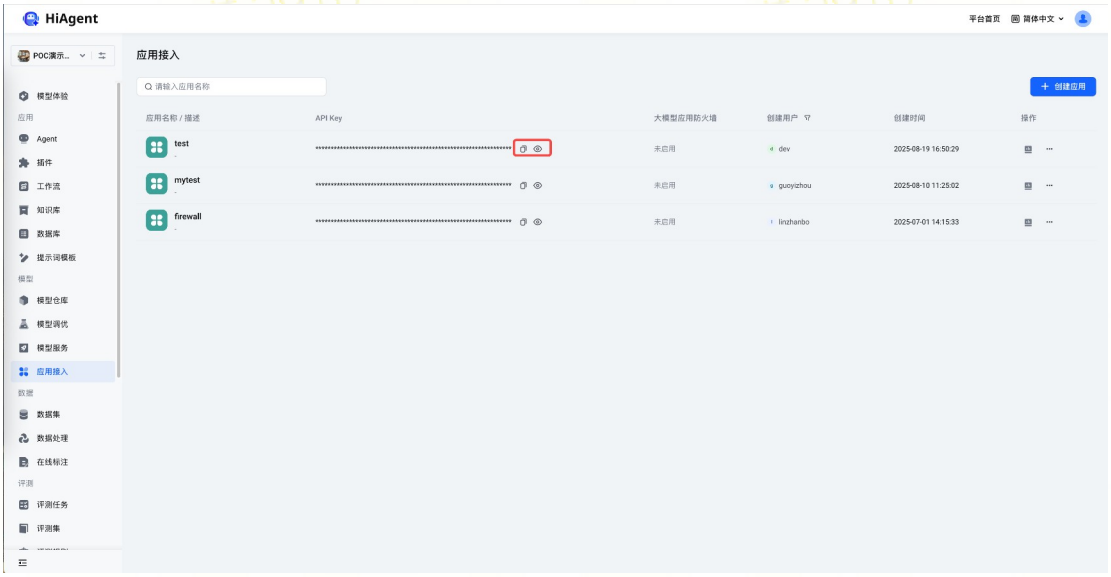
1.7.1 API Key 认证

1. **获取方式**：在 HiAgent 平台创建并配置 API Key。
    1. 创建应用：点击创建应用，输入应用名称和勾选可调用的模型服务，点击确定即可





1. 复制 API Key：在应用列表页查看和复制 API Key



1. **传递方式**：通过 HTTP 请求头 `Authorization` 字段传递，格式为 `Bearer <API_KEY>`。

2. 请求示例

2.1 Python SDK 示例 (大语言模型)

```Python
# pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark

if __name__ == "__main__":
    maas_proxy_svc_host = "{$maas_proxy service 的 nodeip}"
    maas_proxy_svc_port = "{$maas_proxy service 的 nodeport}"
    api_key = "{$应用的 apikey}"
    model_service_id = "{$模型服务 id}"

    client =
Ark(base_url='http://{0}:{1}/api/v3'.format(maas_proxy_svc_host,

maas_proxy_svc_port),
                api_key=api_key, timeout=60)

    print("\n----- standard request -----")
    try:
        response = client.chat.completions.create(
```

```
            model=model_service_id,
            messages=[
                {
                    "role": "user",
                    "content": "给我讲一个小白兔的故事",
                },
            ],
        )
    except Exception as e:
        print("call maas error:", e)
    else:
        print(response.choices[0].message.content)

    print("\n----- stream request -----")

    try:
        stream_response = client.chat.completions.create(
            model=model_service_id,
            messages=[
                {
                    "role": "user",
                    "content": "给我讲一个小白兔的故事",
                },
            ],
            stream=True,
        )
    except Exception as e:
        print("call maas error:", e)
    else:
        for chunk in stream_response:
            if not chunk.choices:
                continue
            print(chunk.choices[0].delta.content, end="")
```

2.2 Python SDK 示例 (Embedding 模型)

```Python
# pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark

if __name__ == "__main__":
    aigw_proxy_svc_host = "{$aigw_node_ip}"
    aigw_proxy_svc_port = "{$aigw_node_port}"
    api_key = "{$app_api_key}"
    model_service_id = "{$model_id}"
```

```
    client =
Ark(base_url='http://{0}:{1}/v1'.format(aigw_proxy_svc_host,

aigw_proxy_svc_port),
                  api_key=api_key, timeout=60)

    try:
        response = client.embeddings.create(
            model=model_service_id,
            input="The food was delicious and the waiter...",
        )
    except Exception as e:
        print("call model error:", e)
    else:
        print(response.data[0].embedding)
```

2.3 Python SDK 示例 (Rerank 模型)

```Python
# pip install 'volcengine-python-sdk[ark]'
import json
from volcenginesdkarkruntime import Ark
from httpx import Response

if __name__ == "__main__":
    aigw_proxy_svc_host = "{$aigw_node_ip}"
    aigw_proxy_svc_port = "{$aigw_node_port}"
    api_key = "{$app_api_key}"
    model_service_id = "{$model_id}"

    client =
Ark(base_url='http://{0}:{1}/v1'.format(aigw_proxy_svc_host,

aigw_proxy_svc_port),
                  api_key=api_key, timeout=60)

    req = {
        "model": model_service_id,
        "query": "A man is eating pasta.",
        "documents": [
            "A man is eating food.",
            "A man is eating a piece of bread.",
            "The girl is carrying a baby.",
            "A man is riding a horse.",
```

```
            "A woman is playing violin."
        ],
        "top_n": 5
    }
    try:
        response = client.post(
            path="rerank",
            body=req,
            cast_to=Response
        )
    except Exception as e:
        print("call model error:", e)
    else:
        result_dict = json.loads(str(response.content, "utf-8"))
        print(result_dict)
```

2.4 Python SDK 示例 (生图模型)

```
Python
# pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark


if __name__ == "__main__":
    aigw_proxy_svc = "http://aigw-proxy:6780"
    api_key = "{$app_api_key}"
    model_service_id = "{$model_id}"

    client = Ark(base_url='http://{0}/v1'.format(aigw_proxy_svc),
                 api_key=api_key, timeout=60)

    imagesResponse = client.images.generate(
        model=model_service_id,
        prompt="a cute cat",

    )

    print(imagesResponse.data[0].url)
```

2.5 Python SDK 示例 (生视频模型)

```
Python
# pip install 'volcengine-python-sdk[ark]'
from volcenginesdkarkruntime import Ark
```

```
import json
from httpx import Response


if __name__ == "__main__":
    aigw_proxy_svc = "http://aigw-proxy:6780"
    api_key = "{$app_api_key}"
    model_service_id = "{$model_id}"

    client = Ark(base_url='http://{0}/v1'.format(aigw_proxy_svc),
                 api_key=api_key, timeout=60)


    req = {
        "model": model_service_id,
        "prompt": "a cute cat",
        "type": "image",
        "images": [
            {
                "url": "xxx.jpg"
            }
        ],
        "ratio": "9:16",
        "seed": -1,
        "duration": 5,
        "resolution": "480p",
    }

    try:
        response = client.post(
            path="/videos/generations/tasks",
            body=req,
            cast_to=Response
        )
    except Exception as e:
        print("call model error:", e)
    else:
        result_dict = json.loads(str(response.content, "utf-8"))
        print(result_dict)
```

2.6 Golang SDK 示例 (大语言模型)

Go

```
// //go:build ignore

package main

import (
        "context"
        "errors"
        "fmt"
        "io"
        "time"


"github.com/volcengine/volcengine-go-sdk/service/arkruntime"

"github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
        "github.com/volcengine/volcengine-go-sdk/volcengine"
)

func main() {
        maasProxySvcHost := "{$maas_proxy service 的 nodeip}"
        maasProxySvcPort := "{$maas_proxy service 的 nodeport}"
        apiKey := "{$应用的 apikey}"
        modelServiceID := "{$模型服务 id}"

        client := arkruntime.NewClientWithApiKey(
                apiKey, arkruntime.WithBaseUrl(
                        fmt.Sprintf("http://%s:%s/api/v3",
maasProxySvcHost, maasProxySvcPort)),
                arkruntime.WithTimeout(60*time.Second))

        ctx := context.Background()

        fmt.Println("\n----- standard request -----")
        request := model.ChatCompletionRequest{
                Model: modelServiceID,
                Messages: []*model.ChatCompletionMessage{
                        {
                                Role: model.ChatMessageRoleUser,
                                Content:
&model.ChatCompletionMessageContent{
                                        StringValue:
volcengine.String("给我讲一个小白兔的故事"),
                                },
```

```
                        },
                },
        }

        resp, err := client.CreateChatCompletion(ctx, request)
        if err != nil {
                fmt.Println(err)
                return
        }
        fmt.Println(*resp.Choices[0].Message.Content.StringValue)

        fmt.Println("\n----- stream request -----")
        streamRequest := model.ChatCompletionRequest{
                Model: modelServiceID,
                Messages: []*model.ChatCompletionMessage{
                        {
                                Role: model.ChatMessageRoleUser,
                                Content:
&model.ChatCompletionMessageContent{
                                        StringValue:
volcengine.String("给我讲一个小白兔的故事"),
                                },
                        },
                },
                Stream: true,
        }

        stream, err := client.CreateChatCompletionStream(ctx,
streamRequest)
        if err != nil {
                fmt.Println(err)
                return
        }
        defer stream.Close() // nolint

        for {
                recv, err := stream.Recv()
                if errors.Is(err, io.EOF) {
                        break
                }
                if err != nil {
                        fmt.Printf("Stream chat error: %v\n", err)
                        return
                }
```

```
                if len(recv.Choices) > 0 {
                        fmt.Print(recv.Choices[0].Delta.Content)
                }
        }
}
```

2.7 Golang SDK 示例 (Embedding 模型)

```Go
package main

import (
        "context"
        "fmt"
        "time"


"github.com/volcengine/volcengine-go-sdk/service/arkruntime"

"github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
)

func main() {
        aigwProxySvcHost := "{$aigw_node_ip}"
        aigwProxySvcPort := "{$aigw_node_port}"
        apiKey := "{$app_api_key}"
        modelServiceID := "{$model_id}"

        client := arkruntime.NewClientWithApiKey(
                apiKey, arkruntime.WithBaseUrl(
                        fmt.Sprintf("http://%s:%s/v1",
aigwProxySvcHost, aigwProxySvcPort)),
                arkruntime.WithTimeout(60*time.Second))

        ctx := context.Background()

        request := model.EmbeddingRequest{
                Model: modelServiceID,
                Input: "The food was delicious and the waiter...",
        }

        resp, err := client.CreateEmbeddings(ctx, request)
        if err != nil {
                return
```

```
        }

        for _, item := range resp.Data {
                fmt.Println(item.Embedding)
        }
}
```

2.8 Golang SDK 示例 (Rerank 模型)

```Go
package main

import (
        "bytes"
        "encoding/json"
        "fmt"
        "io"
        "net/http"

        cohere "github.com/cohere-ai/cohere-go/v2"
)

func main() {
        aigwProxySvcHost := "{$aigw_node_ip}"
        aigwProxySvcPort := "{$aigw_node_port}"
        apiKey := "{$app_api_key}"
        modelServiceID := "{$model_id}"

        req := &cohere.RerankRequest{
                Model: &modelServiceID,
                Query: "A man is eating pasta.",
                Documents: []*cohere.RerankRequestDocumentsItem{
                        {String: "A man is eating food."},
                        {String: "A man is eating a piece of
bread."},
                        {String: "The girl is carrying a baby."},
                        {String: "A man is riding a horse."},
                        {String: "A woman is playing violin."},
                },
        }
        reqBytes, err := json.Marshal(req)
        if err != nil {
                return
        }
```

```
        finalReq, err := http.NewRequest(http.MethodPost,
                fmt.Sprintf("http://%s:%s/v1/rerank",
aigwProxySvcHost, aigwProxySvcPort),
                bytes.NewReader(reqBytes))
        if err != nil {
                fmt.Println(err)
                return
        }
        finalReq.Header.Set("Authorization", fmt.Sprintf("Bearer
%s", apiKey))
        client := &http.Client{}
        resp, err := client.Do(finalReq)
        if err != nil {
                fmt.Println(err)
                return
        }
        defer resp.Body.Close()

        body, err := io.ReadAll(resp.Body)
        if err != nil {
                fmt.Println(err)
                return
        }

        finalResp := make(map[string]interface{})
        if err := json.Unmarshal(body, &finalResp); err != nil {
                fmt.Println(err)
                return
        }

        if resp.StatusCode != 200 {
                fmt.Println(finalResp["message"])
                return
        }

        fmt.Println(finalResp)
}
```

2.9 Golang SDK 示例 (生图模型)

```
Python
package main

import (
```

```
        "context"
        "fmt"
        "time"



"github.com/volcengine/volcengine-go-sdk/service/arkruntime"

"github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"

)

func main() {
        aigwProxySvc := "http://aigw-proxy:6780"
        apiKey := "{$app_api_key}"
        modelServiceID := "d49dkc7933qs73e0qeig"

        client := arkruntime.NewClientWithApiKey(
                apiKey, arkruntime.WithBaseUrl(
                        fmt.Sprintf("http://%s/v1",
aigwProxySvc)),
                arkruntime.WithTimeout(60*time.Second))

        ctx := context.Background()

        generateReq := model.GenerateImagesRequest{
                Model:  modelServiceID,
                Prompt: "a cute cat",

        }

        imagesResponse, err := client.GenerateImages(ctx,
generateReq)
        if err != nil {
                fmt.Printf("generate images error: %v\n", err)
                return
        }

        fmt.Printf("%s\n", *imagesResponse.Data[0].Url)

}
```

2.10 Golang SDK 示例 (生视频模型)

Python

```
package main

import (

        "bytes"
        "encoding/json"
        "fmt"
        "io"
        "net/http"

)

func main() {
        aigwProxySvc := "http://aigw-proxy:6780"
        apiKey := "{$app_api_key}"
        modelServiceID := "d3vju3l8btns738o8sp0"


        req := make(map[string]interface{})
        image := make(map[string]interface{})
        image["url"] = "xxx.jpg"
        images := make([]map[string]interface{}, 0)
        images = append(images, image)
        req["model"] = modelServiceID
        req["prompt"] = "a cute cat"
        req["type"] = "image"
        req["images"] = images
        req["ratio"] = "9:16"
        req["seed"] = -1
        req["duration"] = 5
        req["resolution"] = "480p"

        reqBytes, err := json.Marshal(req)
        if err != nil {
                return
        }

        createReq, err := http.NewRequest(http.MethodPost,

fmt.Sprintf("http://%s/v1/videos/generations/tasks",
aigwProxySvc),
                bytes.NewReader(reqBytes))
        if err != nil {
                fmt.Println(err)
```

```
                return
        }
        createReq.Header.Set("Authorization", fmt.Sprintf("Bearer
%s", apiKey))
        client := &http.Client{}
        resp, err := client.Do(createReq)
        if err != nil {
                fmt.Println(err)
                return
        }
        defer resp.Body.Close()

        body, err := io.ReadAll(resp.Body)
        if err != nil {
                fmt.Println(err)
                return
        }

        createResp := make(map[string]interface{})
        if err := json.Unmarshal(body, &createResp); err != nil {
                fmt.Println(err)
                return
        }

        fmt.Println(createResp)

}
```

2.11 Java SDK 示例 (大语言模型)

```
Java
/*
        <dependency>
            <groupId>com.volcengine</groupId>

<artifactId>volcengine-java-sdk-ark-runtime</artifactId>
            <version>LATEST</version>
        </dependency>
 */

package example_chat_volc;
import com.volcengine.ark.runtime.model.completion.chat.*;
import com.volcengine.ark.runtime.service.ArkService;

import java.time.Duration;
```

```
import java.util.*;

public class example_chat_volc {
    public static void main(String[] args) {

        String maasProxySvcHost = "{$maas_proxy service 的
nodeip}";
        String maasProxySvcPort = "{$maas_proxy service 的
nodeport}";
        String apiKey = "{$应用的 apikey}";
        String modelServiceID = "{$模型服务 id}";

        ArkService service = new
ArkService.Builder().apiKey(apiKey).
                baseUrl(String.format("http://%s:%s/",
maasProxySvcHost, maasProxySvcPort)).
                timeout(Duration.ofSeconds(60)).
                build();

        System.out.println("\n----- standard request -----");
        final List<ChatMessage> messages = new ArrayList<>();
        final ChatMessage userMessage =
ChatMessage.builder().role(ChatMessageRole.USER).content("给我讲一
个小白兔的故事？").build();
        messages.add(userMessage);

        ChatCompletionRequest chatCompletionRequest =
ChatCompletionRequest.builder()
                .model(modelServiceID)
                .messages(messages)
                .build();

        ChatCompletionResult chatCompletionResult =
service.createChatCompletion(chatCompletionRequest);
        chatCompletionResult.getChoices().forEach(
                choice ->
System.out.println(choice.getMessage().getContent())
        );

        System.out.println("\n----- streaming request -----");

        service.streamChatCompletion(chatCompletionRequest)
                .doOnError(Throwable::printStackTrace)
                .blockingForEach(
```

```
                        choice -> {
                            if (choice.getChoices() != null) {

System.out.print(choice.getChoices().get(0).getMessage().getConten
t());
                            }
                        }
                );
    }
}
```

2.12 Java SDK 示例 (Embedding 模型)

```
Java
/*
        <dependency>
            <groupId>com.volcengine</groupId>

<artifactId>volcengine-java-sdk-ark-runtime</artifactId>
            <version>LATEST</version>
        </dependency>
 */

package example_embeddings_volc;
import com.volcengine.ark.runtime.model.embeddings.Embedding;
import
com.volcengine.ark.runtime.model.embeddings.EmbeddingRequest;
import com.volcengine.ark.runtime.service.ArkService;

import java.time.Duration;
import java.util.Collections;
import java.util.List;

public class example_embeddings_volc {
    public static void main(String[] args) {

        String aigwProxySvcHost = "{$aigw_node_ip}";
        String aigwProxySvcPort = "{$aigw_node_port}";
        String apiKey = "{$app_api_key}";
        String modelServiceID = "{$model_id}";

        ArkService service = new
ArkService.Builder().apiKey(apiKey).
                baseUrl(String.format("http://%s:%s/",
aigwProxySvcHost, aigwProxySvcPort)).
```

```
                timeout(Duration.ofSeconds(60)).
                build();

        EmbeddingRequest embeddingRequest =
EmbeddingRequest.builder()
                .model(modelServiceID)
                .input(Collections.singletonList("The food was
delicious and the waiter..."))
                .build();

        List<Embedding> embeddings =
service.createEmbeddings(embeddingRequest).getData();

        System.out.println(embeddings.get(0).getEmbedding());
    }
}
```

2.13 Java SDK 示例 (Rerank 模型)

```
Java
/*
        <dependency>
            <groupId>com.fasterxml.jackson.datatype</groupId>
            <artifactId>jackson-datatype-jdk8</artifactId>
            <version>LATEST</version>
        </dependency>
        <dependency>
            <groupId>com.cohere</groupId>
            <artifactId>cohere-java</artifactId>
            <version>LATEST</version>
        </dependency>
 */
package example_rerank;
import com.cohere.api.requests.RerankRequest;
import com.cohere.api.types.RerankRequestDocumentsItem;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jdk8.Jdk8Module;
import com.squareup.okhttp.MediaType;
import com.squareup.okhttp.OkHttpClient;
import com.squareup.okhttp.Request;
import com.squareup.okhttp.RequestBody;
import com.squareup.okhttp.Response;

import java.io.IOException;
import java.util.List;
```

```
public class example_rerank {
    public static void main(String[] args) {
        try {
            String aigwProxySvcHost = "{$aigw_node_ip}";
            String aigwProxySvcPort = "{$aigw_node_port}";
            String apiKey = "{$app_api_key}";
            String modelServiceID = "{$model_id}";

            RerankRequest request =
RerankRequest.builder().query("A man is eating
pasta.").documents(List.of(
                    RerankRequestDocumentsItem.of("A man is eating
food."),
                    RerankRequestDocumentsItem.of("A man is eating
a piece of bread."),
                    RerankRequestDocumentsItem.of("The girl is
carrying a baby."),
                    RerankRequestDocumentsItem.of("A man is riding
a horse."),
                    RerankRequestDocumentsItem.of("A woman is
playing violin.")
            )).model(modelServiceID).topN(5).build();

            ObjectMapper objectMapper = new ObjectMapper();
            objectMapper.registerModule(new Jdk8Module());
            String jsonInputString =
objectMapper.writeValueAsString(request);

            OkHttpClient client = new OkHttpClient();
            MediaType mediaType =
MediaType.parse("application/json");
            RequestBody body = RequestBody.create(mediaType,
jsonInputString);
            Request finalRequest = new Request.Builder()
                    .url(String.format("http://%s:%s/v1/rerank",
aigwProxySvcHost, aigwProxySvcPort))
                    .post(body)
                    .addHeader("Content-Type", "application/json")
                    .addHeader("Authorization", "Bearer "+ apiKey)
                    .build();
            Response response =
client.newCall(finalRequest).execute();
            System.out.println(response.body().string());
```

```
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

2.14 Java SDK 示例 (生图模型)

```Java


/*
        <dependency>
            <groupId>com.volcengine</groupId>

<artifactId>volcengine-java-sdk-ark-runtime</artifactId>
            <version>LATEST</version>
        </dependency>
 */


package example_vision;

import com.volcengine.ark.runtime.model.content.generation.*;
import
com.volcengine.ark.runtime.model.images.generation.GenerateImagesR
equest;
import
com.volcengine.ark.runtime.model.images.generation.ImagesResponse;
import com.volcengine.ark.runtime.service.ArkService;
import okhttp3.ConnectionPool;
import okhttp3.Dispatcher;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;


public class ImageGenerationsExample {
    public static void main(String[] args) {
        String aigwProxySvc = "http://aigw-proxy:6780";
        String apiKey = "{$app_api_key}";
        String modelServiceID = "{$model_id}";
```

```
        ArkService service = new
ArkService.Builder().apiKey(apiKey).
                baseUrl(String.format("http://%s/",
aigwProxySvc)).
                timeout(Duration.ofSeconds(60)).
                build();

        GenerateImagesRequest generateRequest =
GenerateImagesRequest.builder()
                .model(modelServiceID)
                .prompt("a cute cat")

                .build();

        ImagesResponse imagesResponse =
service.generateImages(generateRequest);

System.out.println(imagesResponse.getData().get(0).getUrl());

        service.shutdownExecutor();

    }
}
```

2.15 Java SDK 示例 (生视频模型)

```
Java


/*
        <dependency>
            <groupId>com.fasterxml.jackson.datatype</groupId>
            <artifactId>jackson-datatype-jdk8</artifactId>
            <version>LATEST</version>
        </dependency>
        <dependency>
            <groupId>com.squareup.okhttp3</groupId>
            <artifactId>okhttp</artifactId>
            <version>LATEST</version>
        </dependency>
        <dependency>
            groupId>com.squareup.okio</groupId>
            <artifactId>okio</artifactId>
            <version>LATEST</version>
        </dependency>
```

```
 */


package example_vision;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jdk8.Jdk8Module;
import com.squareup.okhttp.MediaType;
import com.squareup.okhttp.OkHttpClient;
import com.squareup.okhttp.Request;
import com.squareup.okhttp.RequestBody;
import com.squareup.okhttp.Response;

import java.io.IOException;
import java.util.List;
import java.util.Map;
import java.util.HashMap;
import java.util.ArrayList;


public class ImageGenerationsExample {
    public static void main(String[] args) {
        String aigwProxySvc = "http://aigw-proxy:6780";
        String apiKey = "{$app_api_key}";
        String modelServiceID = "{$model_id}";

        try {

            Map<String, Object> request = new HashMap<>();
            Map<String, Object> image = new HashMap<>();
            image.put("url", "xxx.jpg");
            List<Map<String, Object>> images = new ArrayList<>();
            images.add(image);
            request.put("model", modelServiceID);
            request.put("prompt", "a cute cat");
            request.put("type", "image");
            request.put("images", images);
            request.put("ratio", "9:16");
            request.put("seed", -1);
            request.put("duration", 5);
            request.put("resolution", "480p");

            ObjectMapper objectMapper = new ObjectMapper();
            objectMapper.registerModule(new Jdk8Module());
```

```
            String jsonInputString =
objectMapper.writeValueAsString(request);

            OkHttpClient client = new OkHttpClient();
            MediaType mediaType =
MediaType.parse("application/json");
            RequestBody body = RequestBody.create(mediaType,
jsonInputString);
            Request finalRequest = new Request.Builder()

.url(String.format("http://%s/v1/videos/generations/tasks",
aigwProxySvc))
                    .post(body)
                    .addHeader("Content-Type", "application/json")
                    .addHeader("Authorization", "Bearer "+ apiKey)
                    .build();
            Response response =
client.newCall(finalRequest).execute();
            System.out.println(response.body().string());
        } catch (IOException e) {
            e.printStackTrace();
        }

    }
}
```

2.16 Curl 示例

```
Bash
curl "{maas_proxy_url}" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <ARK_API_KEY>" \
  -d '{
    "model": "doubao-1.5-lite",
    "messages": [{"role": "user", "content": "你好"}]
  }'
```

2.17 响应示例 (大语言模型)

成功响应

```
JSON
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
```

```
  "created": 1681234567,
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "推荐《深度学习》（花书），涵盖神经网络、深度学习理论与应用，适合 AI 入门与进阶。"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 10,
    "completion_tokens": 30,
    "total_tokens": 40
  }
}
```

3. 错误处理

常见错误码

错误类型	HTTP 状态码	说明	处理建议
`InvalidAPIKey`	401	无效 API Key	检查 API Key 是否正确配置
`ModelNotFound`	404	模型未找到	确认模型 ID 是否正确
`MaxTokenExceeded`	413	超过最大 token 限制	减少 `max_tokens` 或缩短上下文
`RateLimitExceeded`	429	请求频率超限	降低调用频率或申请更高配额
`InvalidRequest`	400	请求格式错误	检查参数格式（如 `messages` 格式）

4. 术语表

1. **token**：模型处理的最小文本单元（约 1 个中文字符≈1.5 tokens，1 个英文单词≈1 tokens）。
1. **上下文长度**：模型支持的历史对话+当前输入的总 token 上限。

梁瑾 2918